In [2]:
```python
from IPython.display import display, HTML

display(HTML('''
<style>
.prompt { display:none !important; }
</style>
'''))
```

In [ ]:
```python
import pandas as pd
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import chi2_contingency
from sklearn.model_selection import GridSearchCV, train_test_split
import statsmodels.api as sm
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix, classification_report
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV
from imblearn.over_sampling import SMOTE
from collections import Counter
from sklearn.preprocessing import StandardScaler
from scipy.stats import chi2
```

In [2]:
```python
# Importiamo il dataset
data_set = pd.read_csv("C:/Users/massi/OneDrive/Desktop/Progetto/dataset.csv")
```

In [3]:
```python
data_set.head()
```

Out[3]:

	satisfaction_level	last_evaluation	number_project	average_montly_hours	time_spend_company	Work_accident	left	promotion_last_5years	sales	salary
0	0.38	0.53	2	157	3	0	1	0	sales	low
1	0.80	0.86	5	262	6	0	1	0	sales	medium
2	0.11	0.88	7	272	4	0	1	0	sales	medium
3	0.72	0.87	5	223	5	0	1	0	sales	low
4	0.37	0.52	2	159	3	0	1	0	sales	low

Dimensione del data_set -> numero di righe x numero di colonne

In [4]:
```python
data_set.shape
```

Out[4]:
```
(14999, 10)
```

Esploriamo la struttura del data_set

In [6]:
```python
data_set.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14999 entries, 0 to 14998
Data columns (total 10 columns):
 #   Column                 Non-Null Count  Dtype  
---  ------                 --------------  -----  
 0   satisfaction_level     14999 non-null  float64
 1   last_evaluation        14999 non-null  float64
 2   number_project         14999 non-null  int64  
 3   average_montly_hours   14999 non-null  int64  
 4   time_spend_company     14999 non-null  int64  
 5   Work_accident          14999 non-null  int64  
 6   left                   14999 non-null  int64  
 7   promotion_last_5years  14999 non-null  int64  
 8   sales                  14999 non-null  object 
 9   salary                 14999 non-null  object 
dtypes: float64(2), int64(6), object(2)
memory usage: 1.1+ MB
```

Notiamo che non sono presenti valori mancanti (NA)

Trasformiamo alcune features in tipo categoriale

In [3]:
```python
colonne = ['Work_accident', 'left', 'promotion_last_5years', 'sales', 'salary']

for col in colonne:
    data_set[col] = data_set[col].astype('category')
```

Ci sono duplicati?

In [4]:
```python
print(data_set.duplicated())
```

```
0        False
1        False
2        False
3        False
4        False
         ...  
14994     True
14995     True
14996     True
14997     True
14998     True
Length: 14999, dtype: bool
```

Si ci sono righe duplicate.

Possiamo rimuovere le righe duplicate

In [5]:
```python
data_set.drop_duplicates(inplace = True)
```

Controlliamo di nuovo

In [6]:
```python
print(data_set.duplicated())
```

```
0        False
1        False
2        False
3        False
4        False
         ...  
11995    False
11996    False
11997    False
11998    False
11999    False
Length: 11991, dtype: bool
```

Le righe duplicate sono state eliminate.

Analizziamo e visualizziamo le features presenti nel data_set

In [36]:
```python
data_set['left'].value_counts()
```

Out[36]:
```
0    10000
1     1991
Name: left, dtype: int64
```

I dipendenti che si sono licenziati sono in misura minore rispetto a coloro che non si sono licenziati.

Diamo uno sguardo alle features numeriche, in particolare: average_montly_hours e time_spend_company

In [85]:
```python
data_set[["average_montly_hours", "time_spend_company"]].describe()
```

Out[85]:

	average_montly_hours	time_spend_company
count	11991.000000	11991.000000
mean	200.473522	3.364857
std	48.727813	1.330240
min	96.000000	2.000000
25%	157.000000	3.000000
50%	200.000000	3.000000
75%	243.000000	4.000000
max	310.000000	10.000000

Ora possiamo prendere dei range per creare delle categorie utili

In [7]:
```python
categorie = pd.cut(data_set['time_spend_company'],
                   bins=[0, 2, 5, 10],
                   labels=["basso", "medio", "alto"], right=True)
```

In [8]:
```python
categorie
```

Out[8]:
```
0        medio
1         alto
2        medio
3        medio
4        medio
         ...  
11995     alto
11996     alto
11997     alto
11998     alto
11999    medio
Name: time_spend_company, Length: 11991, dtype: category
Categories (3, object): ['basso' < 'medio' < 'alto']
```

Creazione data_set

In [9]:
```python
data_average_time_spend = pd.DataFrame({
    'average_montly_hours': data_set['average_montly_hours'],
    'time_spend_company': categorie
})
```

time_spend_company vs average_montly_hours

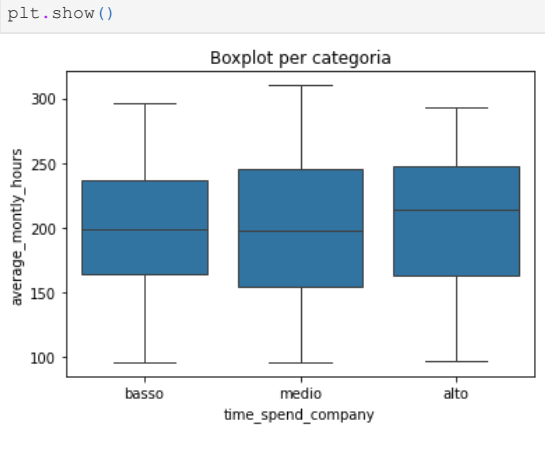
In [10]:
```python
data_average_time_spend.groupby("time_spend_company")[['average_montly_hours']].mean()
```

Out[10]:

time_spend_company	average_montly_hours
basso	199.850859
medio	200.248032
alto	204.932039

I dipendenti che lavorano da più tempo in azienda, in media in un mese lavorano un pò di più.

In [11]:
```python
# Boxplot
sns.boxplot(x='time_spend_company', y='average_montly_hours', data=data_average_time_spend)
plt.title('Boxplot per categoria')
plt.show()
```



Analizziamo il livello di soddisfazione considerando la feature left

In [87]:
```python
data_set.groupby('left')[['satisfaction_level']].mean()
```

Out[87]:

left	satisfaction_level
0	0.667365
1	0.440271

Ovviamente i dipendenti che si licenziano presentano in media un livello di soddisfazione più basso.

Analizziamo il numero di ore mensili di lavoro e il livello di soddisfazione considerando la feature left

In [88]:
```python
data_set.groupby(['left'])[['average_montly_hours', 'satisfaction_level']].mean()
```

Out[88]:

left	average_montly_hours	satisfaction_level
0	198.94270	0.667365
1	208.16223	0.440271

I dipendenti che si licenziano presentano non solo in media un livello di soddisfazione più basso, ma presentano in media un numero di ore di lavoro mensili maggiore.

Analizziamo ora la feature number_project considerando la feature left

In [18]:
```python
data_set.groupby('left')[['number_project']].mean()
```

Out[18]:

left	number_project
0	3.786800
1	3.883476

In media i dipendenti che si licenziano e quelli che non si licenziano partecipano allo stesso numero di progetti.

Analizziamo la feature last_evaluation considerando la feature left

In [19]:
```python
data_set.groupby('left')[['last_evaluation']].mean()
```

Out[19]:

left	last_evaluation
0	0.715667
1	0.721783

In media i dipendenti che si licenziano e quelli che non si licenziano presentano la stessa last_evaluation (ultima valutazione)

Applichiamo una mask e selezioniamo solo i dipendenti che si sono licenziati

In [20]:
```python
mask = data_set['left']==1
```

Summary delle variabili numeriche

In [22]:
```python
data_set[mask].describe()
```

Out[22]:

	satisfaction_level	last_evaluation	number_project	average_montly_hours	time_spend_company
count	1991.000000	1991.000000	1991.000000	1991.000000	1991.000000
mean	0.440271	0.721783	3.883476	208.162230	3.881467
std	0.265207	0.197436	1.817139	61.295145	0.974041
min	0.090000	0.450000	2.000000	126.000000	2.000000
25%	0.110000	0.520000	2.000000	146.000000	3.000000
50%	0.410000	0.790000	4.000000	226.000000	4.000000
75%	0.730000	0.910000	6.000000	262.500000	5.000000
max	0.920000	1.000000	7.000000	310.000000	6.000000

Selezioniamo solo i dipendenti che non si sono licenziati e applichiamo il summary alle variabili numeriche

In [23]:
```python
data_set[~mask].describe()
```

Out[23]:

	satisfaction_level	last_evaluation	number_project	average_montly_hours	time_spend_company
count	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	0.667365	0.715667	3.786800	198.942700	3.262000
std	0.217082	0.161919	0.981755	45.665507	1.367239
min	0.120000	0.360000	2.000000	96.000000	2.000000
25%	0.540000	0.580000	3.000000	162.000000	2.000000
50%	0.690000	0.710000	4.000000	198.000000	3.000000
75%	0.840000	0.850000	4.000000	238.000000	4.000000
max	1.000000	1.000000	6.000000	287.000000	10.000000

Analizzando il summary delle variabili numeriche sembra che (come visto in precedenza) effettivamente i dipendenti che si licenziano presentano in media un numero di ore lavorative mensili maggiore.

Analisi bivariata tra le variabili categoriali

In [65]:
```python
def cramers_v(confusion_matrix):
    """Calcola Cramér's V da una tabella di contingenza."""
    chi2, _, _, _ = chi2_contingency(confusion_matrix)
    n = confusion_matrix.sum().sum()
    min_dim = min(confusion_matrix.shape) - 1
    return np.sqrt(chi2 / (n * min_dim)) if min_dim > 0 else 0.0

def cramers_v_matrix(data, x_cat_vars):
    """
    Calcola la matrice di Cramér's V tra tutte le variabili categoriali nella lista x_cat_vars.

    Parameters:
        data (pd.DataFrame): dataset contenente le variabili.
        x_cat_vars (list): lista delle variabili categoriche.

    Returns:
        pd.DataFrame: matrice quadrata con i valori di Cramér's V.
    """
    n = len(x_cat_vars)
    matrix = pd.DataFrame(np.zeros((n, n)), index=x_cat_vars, columns=x_cat_vars)

    for i in range(n):
        for j in range(i, n):
            if i == j:
                matrix.iloc[i, j] = 1.0  # Cramér's V massimo con sé stessa
            else:
                # Crea tabella di contingenza tra le due variabili
                contingency_table = pd.crosstab(data[x_cat_vars[i]], data[x_cat_vars[j]])
                v = cramers_v(contingency_table)
                matrix.iloc[i, j] = v
                matrix.iloc[j, i] = v  # simmetrico

    return matrix
```

In [66]:
```python
x_cat_vars = ['sales', 'salary', 'promotion_last_5years', 'Work_accident']
cramer_matrix = cramers_v_matrix(data_set, x_cat_vars)
print(cramer_matrix)
```

```
                          sales    salary  promotion_last_5years  \
sales                  1.000000  0.110249               0.122931   
salary                 0.110249  1.000000               0.088227   
promotion_last_5years  0.122931  0.088227               1.000000   
Work_accident          0.025677  0.003686               0.028957   

                       Work_accident  
sales                       0.025677  
salary                      0.003686  
promotion_last_5years       0.028957  
Work_accident               1.000000  
```

L'indice di Cramer per le varie coppie di variabili categoriali presenta valori bassi, quindi la dipendenza tra le variabili categoriali è trascurabile.

Analisi delle variabili categoriali in relazione alla variabile left

In [31]:
```python
def test_indipendenza_vs_left(data, cat_vars, target='left', alpha=0.05):
    """
    Testa l'indipendenza di ogni variabile categorica rispetto a 'left' usando il chi-quadrato.

    Parameters:
        data (pd.DataFrame): DataFrame con i dati.
        cat_vars (list): Lista di variabili categoriche da testare contro 'left'.
        target (str): Nome della variabile target (default 'left').
        alpha (float): Livello di significatività (default 0.05).
    """
    indipendenti = []
    dipendenti = []

    for var in cat_vars:
        if var == target:
            continue  # salto la variabile target stessa

        contingency_table = pd.crosstab(data[var], data[target])
        chi2, p, _, _ = chi2_contingency(contingency_table)

        if p < alpha:
            dipendenti.append(var)
        else:
            indipendenti.append(var)

    print(f"Al livello di confidenza {(1-alpha)*100}%:")
    print(f"Variabili dipendenti da '{target}': {dipendenti}")
    print(f"Variabili indipendenti da '{target}': {indipendenti}")

# Esempio di utilizzo:
cat_vars = ['sales', 'salary', 'promotion_last_5years', 'Work_accident', 'left']
test_indipendenza_vs_left(data_set, cat_vars)
```

```
Al livello di confidenza 95.0%:
Variabili dipendenti da 'left': ['sales', 'salary', 'promotion_last_5years', 'Work_accident']
Variabili indipendenti da 'left': []
```

Con il test chi-quadrato, abbiamo concluso che tra le variabili categoriali e la variabile left esiste un'associazione, dunque possiamo pensare di utilizzare tali variabili all'interno di un dato modello per prevedere il comportamento della variabile left.

FINE ANALISI ESPLORATIVA

INIZIO SECONDA PARTE

La variabile dipendente left assume due valori: 0 ed 1. Dobbiamo classificare in base alle features disponibili se un un dipendente ha richiesto o meno il licenziamento.

Probelma di classificazione, 2 classi.

MODELLI STATISTICI: REGRESSIONE LOGISTICA, ALBERO DI DECISIONE

In [71]:
```python
# PRIMO MODELLO: REGRESSIONE LOGISTICA

# Trasformiamo le variabili 'sales', 'salary', 'promotion_last_5years', 'Work_accident' attraverso la codifica dell' one-hot
# encoding

# Seleziono le variabili categoriali

# 1. Trova tutte le colonne categoriche (object o category)
categorical_cols = data_set.select_dtypes(include=['category', 'object']).columns

# 2. Escludo la variabile target 'left'
categorical_cols = [col for col in categorical_cols if col != 'left']

# 3. Applico il One Hot Encoding solo alle categoriche (escludendo 'left'), con drop_first=True
data_set_encoded = pd.get_dummies(data_set, columns=categorical_cols, drop_first=True)

# 4. La variabile 'left' deve essere numerica 0-1
data_set_encoded['left'] = data_set_encoded['left'].astype(int)
```

Ora dividiamo il data_set in training set e test set, in particolare il 75% delle osservazioni faranno parte del training set mentre il restante 25% farà parte del test set.

In [72]:
```python
y = data_set_encoded['left']

X = data_set_encoded.drop('left', axis=1)

X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y,test_size=0.25, random_state=42)
```

In [73]:
```python
# Training set
X_train.head()
```

Out[73]:

	satisfaction_level	last_evaluation	number_project	average_montly_hours	time_spend_company	Work_accident_1	promotion_last_5years_1	sales_RandD	sales_accounting	sales_hr	sales_management	sales_marketing
984	0.75	0.86	5	260	5	0	0	0	0	0	0	0
610	0.09	0.95	7	256	4	0	0	0	0	0	0	1
7033	0.68	0.61	3	134	4	0	0	0	0	0	0	0
6411	0.59	0.75	4	190	2	0	0	0	0	0	0	0
8388	0.59	0.79	3	217	4	0	0	0	0	0	0	0

In [75]:
```python
# Test set
X_test
```

Out[75]:

	satisfaction_level	last_evaluation	number_project	average_montly_hours	time_spend_company	Work_accident_1	promotion_last_5years_1	sales_RandD	sales_accounting	sales_hr	sales_management	sales_marketing
11098	0.69	0.83	4	252	3	0	0	0	0	0	1	0
5472	0.19	0.64	5	181	4	1	0	0	0	0	0	0
7367	0.79	0.78	2	228	2	0	0	0	0	0	0	0
3605	0.73	0.81	4	245	2	0	0	0	0	0	0	0
3248	0.76	0.62	3	150	2	1	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...
8511	1.00	0.60	4	261	3	0	0	0	0	0	0	0
11933	0.95	0.99	5	251	4	0	0	0	0	0	0	0
3283	0.54	0.61	3	182	2	0	0	0	0	0	0	0
6917	0.75	0.86	6	114	4	0	0	0	0	0	0	0
5640	0.21	0.76	4	165	6	1	0	0	0	0	0	0

2998 rows × 18 columns

In [77]:
```python
# Aggiungo la colonna con gli 1, intercetta

X_train = sm.add_constant(X_train)

X_test = sm.add_constant(X_test)

# Fit sul train -> Addestramento del modello di regressione logistica

model = sm.Logit(y_train, X_train)

result = model.fit()
```

```
Optimization terminated successfully.
         Current function value: 0.358998
         Iterations 7
```

In [84]:
```python
print(result.summary())
```

```
                           Logit Regression Results                           
==============================================================================
Dep. Variable:                   left   No. Observations:                 8993
Model:                          Logit   Df Residuals:                     8974
Method:                           MLE   Df Model:                           18
Date:                Wed, 02 Jul 2025   Pseudo R-squ.:                  0.2014
Time:                        14:07:35   Log-Likelihood:                -3228.5
converged:                       True   LL-Null:                       -4042.5
Covariance Type:            nonrobust   LLR p-value:                     0.000
===========================================================================================
                              coef    std err          z      P>|z|      [0.025      0.975]
-------------------------------------------------------------------------------------------
const                      -2.3090      0.287     -8.036      0.000      -2.872      -1.746
satisfaction_level         -3.9478      0.140    -28.264      0.000      -4.222      -3.674
last_evaluation             0.6504      0.207      3.141      0.002       0.245       1.056
number_project             -0.2901      0.030     -9.767      0.000      -0.348      -0.232
average_montly_hours        0.0041      0.001      5.734      0.000       0.003       0.005
time_spend_company          0.3162      0.023     13.487      0.000       0.270       0.362
Work_accident_1            -1.3995      0.127    -11.027      0.000      -1.648      -1.151
promotion_last_5years_1    -1.0861      0.414     -2.624      0.009      -1.897      -0.275
sales_RandD                -0.4499      0.186     -2.418      0.016      -0.815      -0.085
sales_accounting            0.0387      0.175      0.221      0.825      -0.305       0.382
sales_hr                    0.0817      0.175      0.466      0.641      -0.262       0.425
sales_management           -0.2812      0.230     -1.225      0.221      -0.731       0.169
sales_marketing             0.0477      0.174      0.274      0.784      -0.293       0.389
sales_product_mng          -0.0630      0.173     -0.364      0.716      -0.402       0.276
sales_sales                 0.0650      0.126      0.515      0.606      -0.182       0.312
sales_support              -0.0003      0.138     -0.002      0.998      -0.270       0.270
sales_technical             0.0948      0.132      0.716      0.474      -0.165       0.354
salary_low                  1.9910      0.198     10.052      0.000       1.603       2.379
salary_medium               1.5831      0.199      7.957      0.000       1.193       1.973
===========================================================================================
```

Commento variabili numeriche

satisfaction_level -> coef -3.9478

Possiamo dire che, ceteris paribus, un incremento unitario nella soddisfazione del dipendente riduce l’odds di y=1 (licenziamento) di un fattore di circa 0.019 (exp(-3.9478)). Più il dipendente è soddisfatto menò sarà probabile il licenziamento.

average_montly_hours -> coef 0.0041

Possiamo dire che, ceteris paribus, un incremento unitario del numero di ore mensili lavorative, aumenta l'odds di y=1 (licenziamento) di un fattore di circa 1.004 (exp(0.0041)).

Commento variabili categoriali

promotion_last_5years_1 -> coef -1.0861. La categoria base è 0,ovvero quei dipendenti che negli ultimi 5 anni non hanno ricevuto una promozione. Ceteris paribus, i dipendenti che hanno ricevuto una promozione negli ultimi 5 anni presentano una riduzione dell' odds di y=1 (licenziamento) di un fattore che è pari a exp(-1.0861), ovvero 0.33 rispetto ai dipendenti che non hanno ricevuto una promozione negli ultimi 5 anni.

In [79]:
```python
# Previsione sul test set
pred_probs = result.predict(X_test)

# Se la previsione è > 0.5, allora classifico l'osservazione i-esima del test set come 1, ovvero un dipendente che ha richiesto
# il licenziamento.

pred_class = (pred_probs >= 0.5).astype(int)
```

In [80]:
```python
# Visualizzo le previsioni
pred_class
```

Out[80]:
```
11098    0
5472     0
7367     0
3605     0
3248     0
        ..
8511     0
11933    0
3283     0
6917     0
5640     0
Length: 2998, dtype: int32
```

In [82]:
```python
# Calcolo la matrice di confusione
cm = confusion_matrix(y_test, pred_class)

# Stampa la matrice di confusione numerica
print("Matrice di confusione:")
print(cm)
```

```
Matrice di confusione:
[[2394  106]
 [ 395  103]]
```

Sulla diagonale principale abbiamo le osservazioni correttamente classificate.

2394 -> I dipendenti che non si sono licenziati e che il modello di regressione logistica ha classificato correttamente.

103 -> I dipendenti che si sono licenziati e che il modello di regressione logistica ha classificato correttamente.

395 -> I dipendenti che si sono licenziati e che il modello di regressione logistica ha classificato come non licenziati.

106 -> I dipendenti che non si sono licenziati e che il modello di regressione logistica ha classificato come licenziati.

In [89]:
```python
# Metriche di performance
accuracy = accuracy_score(y_test, pred_class)
precision = precision_score(y_test, pred_class)
recall = recall_score(y_test, pred_class)
f1 = f1_score(y_test, pred_class)
roc_auc = roc_auc_score(y_test, pred_class)

print(f"Accuracy: {accuracy:.3f}")
print(f"Precision: {precision:.3f}")
print(f"Recall (Sensibilità): {recall:.3f}")
print(f"F1 Score: {f1:.3f}")
print(f"AUC ROC: {roc_auc:.3f}")
```

```
Accuracy: 0.833
Precision: 0.493
Recall (Sensibilità): 0.207
F1 Score: 0.291
AUC ROC: 0.582
```

Commento metriche di performance

L'accuratezza ci dice che 83/100 volte il classificatore classifica correttamente l'osservazione i-esima. 83% di accuratezza.

Recall -> 0.207 Rappresenta il rapporto tra VERI POSITIVI / (VERI POSITIVI + FALSI NEGATIVI) = 103 / (103+395), rappresenta la frazione di (veri) positivi correttamente classificati. Per veri positivi si intende i dipendenti che veramente si sono licenziati.

Precision -> 0.493 Rappresenta il rapporto tra VERI POSITIVI e i POSITIVI classificati dal modello di regressione logistica, quindi 103 / (103+106). Rappresenta una misura di esattezza/fedeltà del classificatore.

F1 Score -> 0.291 Rappresenta una "media armonica" di Precision e Recall. Un alto valore di F1 indica una buona performance.

Dalle metriche di performance possiamo notare come il classificatore in realtà non sia cosi preciso nel classificare correttamente i veri positivi, ovvero i dipendenti che veramente si sono licenziati.

ORA, PROVIAMO UN MODELLO ALTERNATIVO ALLA REGRESSIONE LOGISTICA

ALBERO DI DECISIONE

NELLA PRIMA FASE, CHIAMATA MODEL SELECTION, SCEGLIAMO IL MODELLO "CAMPIONE", OVVERO QUEL MODELLO CHE IN MEDIA PRESENTA LE MIGLIORI PERFORMANCE PREDITTIVE.

NELLA SECONDA FASE, CHIAMATA MODEL VALIDATION, UTILIZZEREMO IL MODELLO SCELTO IN FASE DI SELEZIONE.

In [113…
```python
dt = DecisionTreeClassifier(random_state=42)


param_grid = {
    'max_depth': [3, 5, 10, None],        # profondità massima dell'albero
    'min_samples_split': [2, 5, 10],      # min. campioni per split
    'min_samples_leaf': [1, 2, 4],        # min. campioni per foglia
    'criterion': ['gini', 'entropy']      # funzione di split
}

grid_search = GridSearchCV(estimator=dt,
                           param_grid=param_grid,
                           cv=5,
                           scoring='accuracy',  # metrica per scegliere il migliore
                           n_jobs=-1)
```

PARAM GRID RAPPRESENTA LA GRIGLIA DEGLI IPERPARAMETRI COMUNI A UN ALBERO DI DECISIONE. OGNI COMBINAZIONE DI IPERPARAMETRI SARÀ

PRESA IN CONSIDERAZIONE. PER UNA DATA COMBINAZIONE DI IPERPARAMETRI, IN FASE DI MODEL SELECTION APPLICHEREMO

LA K-FOLD CROSS VALIDATION CON K=5. IL K-ESIMO FOLD VIENE CONSIDERATO COME IL FOLD DI VALIDAZIONE MENTRE I RESTANTI K-1 VERRANNO

UTILIZZATI DALL'ALBERO DI DECISIONE PER L'ADDESTRAMENTO (STIMA). LA VALIDAZIONE SUL K-ESIMO FOLD VIENE FATTA PRENDENDO IN

CONSIDERAZIONE COME METRICA, L'ACCURATEZZA, DUNQUE ALLA FINE, AVREMO 5 METRICHE DI PERFORMANCE, OVVERO 5 "ACCURACY". INFINE

SI CALCOLA L'ACCURACY MEDIA.

QUESTO VIENE FATTO PER OGNI COMBINAZIONE DI IPERPARAMETRI, IN MODO TALE DA AVERE 1/2/3 MODELLI DA PRENDERE IN CONSIDERAZIONE.

In [117…
```python
# fit sui dati di training (X_train, y_train)
grid_search.fit(X_train, y_train)

# Stampa i migliori iperparametri
print("Best parameters:", grid_search.best_params_)
```

```
Best parameters: {'criterion': 'gini', 'max_depth': 10, 'min_samples_leaf': 2, 'min_samples_split': 10}
```

Il modello scelto presenta le seguenti caratteristiche: Profondità dell'albero -> 10, min_samples_split: 10 -> ovvero il numero minimo di osservazioni per cui decidere effettivamente se fare un'ulteriore split o meno è 10 osservazioni, min_samples_leaf: 2 -> Il numero di osservazioni minimo presenti nelle foglie è 2.

Criterio utilizzato per valutare l'impurità delle foglie, dato uno split -> GINI.

In [118…
```python
# Utilizziamo il miglior modello
best_dt = grid_search.best_estimator_
```

In [126…
```python
# MODEL VALIDATION

# Previsioni sul test set con il miglior modello
y_pred = best_dt.predict(X_test)

# Matrice di confusione
cm = confusion_matrix(y_test, y_pred)
print("Matrice di confusione:")
print(cm)
```

```
Matrice di confusione:
[[2484   16]
 [  36  462]]
```

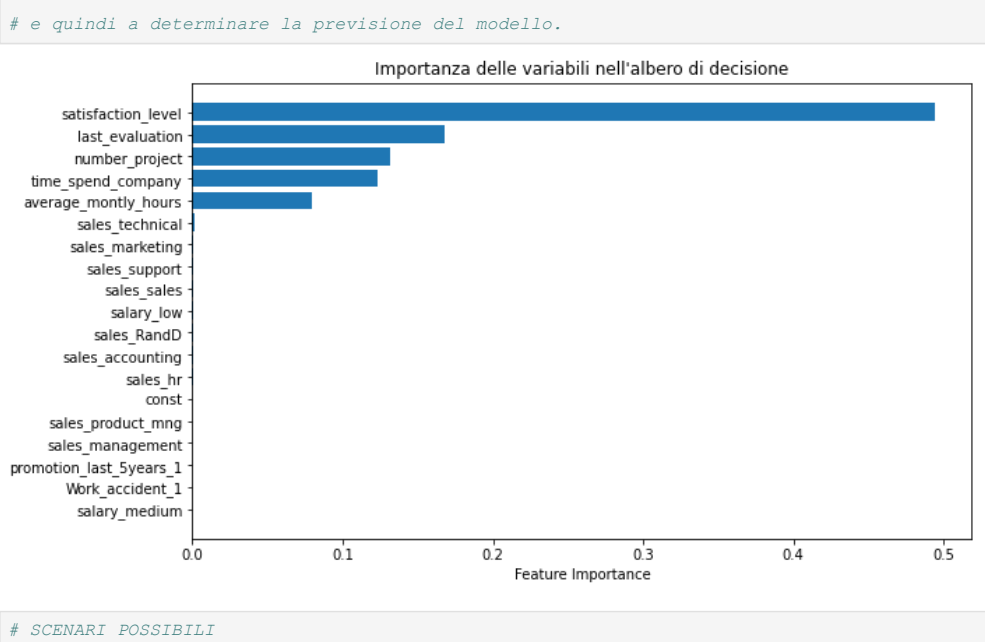
In [125…
```python
# Metriche di performance
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_pred)

print(f"Accuracy: {accuracy:.3f}")
print(f"Precision: {precision:.3f}")
print(f"Recall (Sensibilità): {recall:.3f}")
print(f"F1 Score: {f1:.3f}")
print(f"AUC ROC: {roc_auc:.3f}")
```

```
Accuracy: 0.983
Precision: 0.967
Recall (Sensibilità): 0.928
F1 Score: 0.947
AUC ROC: 0.961
```

Tutte le metriche di performance dell'albero di decisione sono nettamente superiori rispetto alle metriche di performance della regressione logistica.

TRA L'ALBERO DI DECISIONE E IL CLASSIFICATORE CHE SCATURISCE DALLA REGRESSIONE LOGISTICA, SCEGLIAMO L'ALBERO DI DECISIONE.

In [143…
```python
# Importanza delle feature
importances = best_dt.feature_importances_

# Prendi i nomi delle feature usate (es. colonne del DataFrame di training)
feature_names = X_train.columns  # o X_albero_decisione_train.columns se usi quello

# Crea un DataFrame per visualizzarle meglio
df_importance = pd.DataFrame({
    'feature': feature_names,
    'importance': importances
})

# Ordina per importanza decrescente
df_importance = df_importance.sort_values(by='importance', ascending=False)

plt.figure(figsize=(10,6))
plt.barh(df_importance['feature'], df_importance['importance'])
plt.gca().invert_yaxis()  # per mettere la feature più importante sopra
plt.xlabel("Feature Importance")
plt.title("Importanza delle variabili nell'albero di decisione")
plt.show()

# Satisfaction_level, last_evaluation e number_project sono le 3 feature che contribuiscono a suddividere i dati

# e quindi a determinare la previsione del modello.
```



In [136…
```python
# SCENARI POSSIBILI

nuovo_campione_dict = {
    'const': [1, 1],
    'satisfaction_level': [0.6, 0.2],
    'last_evaluation': [0.5, 0.1],
    'number_project': [3, 5],
    'average_montly_hours': [200, 300],
    'time_spend_company': [4, 6],
    'Work_accident_1': [1, 1],
    'promotion_last_5years_1': [0, 0],
    'sales_RandD': [0, 0],
    'sales_accounting': [0, 0],
    'sales_hr': [0, 0],
    'sales_management': [0, 0],
    'sales_marketing': [0, 1],
    'sales_product_mng': [0, 0],
    'sales_sales': [1, 0],
    'sales_support': [0, 0],
    'sales_technical': [0, 0],
    'salary_low': [0, 1],
    'salary_medium': [1, 0],
}

# Creo DataFrame

df_nuovo_campione = pd.DataFrame(nuovo_campione_dict)

# Previsione

previsione = best_dt.predict(df_nuovo_campione)

print("Previsione:", previsione) # [No Licenziamento - Licenziamento]
```

```
Previsione: [0 1]
```

In [140…
```python
best_dt.predict_proba(df_nuovo_campione)
```

Out[140]:
```
array([[1., 0.],
       [0., 1.]])
```

Nel primo caso la classe assegnata è "No Licenziamento" con probabilità 1.

Nel secondo caso la classe assegnata è "Licenziamento" con probabilità 1.

NOTA: LE CLASSI 0-1 (NO LICENZIAMENTO-LICENZIAMENTO) SONO SBILANCIATE

In [144…
```python
data_set['left'].value_counts()
```

Out[144]:
```
0    10000
1     1991
Name: left, dtype: int64
```

LO SBILANCIAMENTO DELLE CLASSI PUÒ INFLUIRE SUL CLASSIFICATORE IN QUANTO GRAZIE AL MAGGIOR NUMERO DI ESEMPI (IN QUESTO CASO DIPENDENTI CHE NON SI SONO LICENZIATI) IL CLASSIFICATORE PUÒ FARE PIU FATICA A RICONOSCERE LA CLASSE MINORITARIA, IN QUESTO CASO I DIPENDENTI CHE SI SONO LICENZIATI.

Il rischio è che il classificatore diventi poco sensibile alla classe minoritaria, riducendo la capacità di riconoscere chi effettivamente si licenzia.

UNA TECNICA CHE PUO ESSERE UTILIZZATA -> SMOTE

In [151…
```python
print("Distribuzione train prima SMOTE:", Counter(y_train))
print("Distribuzione test:", Counter(y_test))
```

```
Distribuzione train prima SMOTE: Counter({0: 7500, 1: 1493})
Distribuzione test: Counter({0: 2500, 1: 498})
```

In [152…
```python
# Applico SMOTE solo sul training set

smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)

print("Distribuzione train dopo SMOTE:", Counter(y_train_res))
```

```
Distribuzione train dopo SMOTE: Counter({1: 7500, 0: 7500})
```

IN QUESTO CASO, VENGONO GENERATI ESEMPI SINTETICI DELLA CLASSE MINORITARIA GRAZIE ALL'INTERPOLAZIONE DI ESEMPI REALI VICINI NELLO SPAZIO DELLE FEATURE

CONSIDERIAMO IL MODELLO DI REGRESSIONE LOGISTICA, INOLTRE QUESTA VOLTA, STANDARDIZZIAMO LE FEATURES NUMERICHE

In [162…
```python
# Selezioniamo solo le colonne numeriche

numerical_cols = X_train_res.select_dtypes(include=['int64', 'float64']).columns
col_intercetta = 'const'
num_cols = [col for col in numerical_cols if col != col_intercetta]
num_cols

# L'intercetta non la considero
```

Out[162]:
```
['satisfaction_level',
 'last_evaluation',
 'number_project',
 'average_montly_hours',
 'time_spend_company']
```

In [164…
```python
# STANDARDIZZO
scaler = StandardScaler()
X_train_res[num_cols] = scaler.fit_transform(X_train_res[num_cols])
```

In [165…
```python
X_train_res
```

Out[165]:

	const	satisfaction_level	last_evaluation	number_project	average_montly_hours	time_spend_company	Work_accident_1	promotion_last_5years_1	sales_RandD	sales_accounting	sales_hr	sales_management
0	1.0	0.744874	0.785917	0.858292	1.042625	1.230485	0	0	0	0	0	0
1	1.0	-1.746943	1.288474	2.268100	0.969075	0.389147	0	0	0	0	0	0
2	1.0	0.480590	-0.610075	-0.551517	-1.274194	0.389147	0	0	0	0	0	0
3	1.0	0.140797	0.171681	0.153387	-0.244497	-1.293529	0	0	0	0	0	0
4	1.0	0.140797	0.395040	-0.551517	0.251964	0.389147	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...
14995	1.0	1.075116	0.946705	0.858292	0.766813	1.230485	0	0	0	0	0	0
14996	1.0	-0.501035	-1.347425	-1.256422	-1.366131	-0.452191	0	0	0	0	0	0
14997	1.0	1.144980	1.405340	0.153387	1.189725	1.230485	0	0	0	0	0	0
14998	1.0	-1.691546	0.909076	1.563196	1.649411	0.389147	0	0	0	0	0	0
14999	1.0	-0.203112	-0.788428	-1.256422	0.840363	0.389147	0	0	0	0	0	0

15000 rows × 19 columns

In [167…
```python
# FASE DI STIMA
model_2 = sm.Logit(y_train_res, X_train_res)

result = model_2.fit()
```

```
Optimization terminated successfully.
         Current function value: 0.427543
         Iterations 7
```

In [169…
```python
# PREVISIONE

# ANCHE QUI STANDARDIZZIAMO LE VARIABILI NUMERICHE PER X_test (SMOTE NON SI APPLICA AL TEST SET)

numerical_cols = X_test.select_dtypes(include=['int64', 'float64']).columns
col_intercetta = 'const'
num_cols = [col for col in numerical_cols if col != col_intercetta]
num_cols
```

Out[169]:
```
['satisfaction_level',
 'last_evaluation',
 'number_project',
 'average_montly_hours',
 'time_spend_company']
```

In [170…
```python
X_test[num_cols] = scaler.fit_transform(X_test[num_cols])
```

In [171…
```python
X_test
```

Out[171]:

	const	satisfaction_level	last_evaluation	number_project	average_montly_hours	time_spend_company	Work_accident_1	promotion_last_5years_1	sales_RandD	sales_accounting	sales_hr	sales_management
11098	1.0	0.240193	0.689636	0.177620	1.080603	-0.259124	0	0	0	0	0	1
5472	1.0	-1.831080	-0.443355	1.043482	-0.372952	0.494371	1	0	0	0	0	0
7367	1.0	0.654448	0.391481	-1.554104	0.589260	-1.012619	0	0	0	0	0	0
3605	1.0	0.405895	0.570374	0.177620	0.937294	-1.012619	0	0	0	0	0	0
3248	1.0	0.530171	-0.562617	-0.688242	-1.007603	-1.012619	1	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...
8511	1.0	1.524382	-0.681879	0.177620	1.264856	-0.259124	0	0	0	0	0	0
11933	1.0	1.317255	1.643734	1.043482	1.060130	0.494371	0	0	0	0	0	0
3283	1.0	-0.381189	-0.622248	-0.688242	-0.352480	-1.012619	0	0	0	0	0	0
6917	1.0	0.488746	0.868530	1.909344	-1.744617	0.494371	0	0	0	0	0	0
5640	1.0	-1.748229	0.272219	0.177620	-0.700514	2.001362	1	0	0	0	0	0

2998 rows × 19 columns

In [172…
```python
# PREVISIONE
# Previsione sul test set
pred_probs = result.predict(X_test)

# Se la previsione è > 0.5, allora classifico l'osservazione i-esima del test set come 1, ovvero un dipendente che ha richiesto
# il licenziamento.

pred_class = (pred_probs >= 0.5).astype(int)
```

In [173…
```python
pred_class
```

Out[173]:
```
11098    0
5472     0
7367     0
3605     0
3248     0
        ..
8511     0
11933    0
3283     1
6917     0
5640     1
Length: 2998, dtype: int32
```

In [174…
```python
# Calcolo la matrice di confusione
cm = confusion_matrix(y_test, pred_class)

# Stampa la matrice di confusione numerica
print("Matrice di confusione:")
print(cm)
```

```
Matrice di confusione:
[[1816  684]
 [ 132  366]]
```

In [175…
```python
# Metriche di performance
accuracy = accuracy_score(y_test, pred_class)
precision = precision_score(y_test, pred_class)
recall = recall_score(y_test, pred_class)
f1 = f1_score(y_test, pred_class)
roc_auc = roc_auc_score(y_test, pred_class)

print(f"Accuracy: {accuracy:.3f}")
print(f"Precision: {precision:.3f}")
print(f"Recall (Sensibilità): {recall:.3f}")
print(f"F1 Score: {f1:.3f}")
print(f"AUC ROC: {roc_auc:.3f}")
```

```
Accuracy: 0.728
Precision: 0.349
Recall (Sensibilità): 0.735
F1 Score: 0.473
AUC ROC: 0.731
```

QUESTI RISULTATI MOSTRANO CHE, NONOSTANTE UNA LEGGERA RIDUZIONE DELL’ACCURACY E DELLA PRECISION, L’USO DI SMOTE ABBINATO ALLA STANDARDIZZAZIONE HA PERMESSO UN NOTEVOLE MIGLIORAMENTO DELLA CAPACITÀ DEL MODELLO DI RICONOSCERE LA CLASSE MINORITARIA, COME DIMOSTRATO DALL’AUMENTO DEL RECALL E DELL'AUC.

IN PARTICOLARE, L'AUMENTO DELL'F1 SCORE INDICA UN MIGLIOR EQUILIBRIO TRA PRECISION E RECALL, RENDENDO IL MODELLO PIÙ EFFICACE NEL RICONOSCIMENTO DELLE CLASSI SBILANCIATE. TUTTAVIA L'ALBERO DI DECISIONE STIMATO IN PRECEDENZA RAPPRESENTA LA SOLUZIONE MIGLIORE.

ULTIMA CONSIDERAZIONE: SI PUÒ PENSARE DI CONFRONTARE UN MODELLO COMPLETO CON UN MODELLO RIDOTTO

In [189…
```python
feature_to_exclude = ['salary_low', 'salary_medium']
features_ridotte = [col for col in X_train_res.columns if col not in feature_to_exclude]

X_train_res_red = X_train_res[features_ridotte]
X_train_res_red
```

Out[189]:

	const	satisfaction_level	last_evaluation	number_project	average_montly_hours	time_spend_company	Work_accident_1	promotion_last_5years_1	sales_RandD	sales_accounting	sales_hr	sales_management
0	1.0	0.744874	0.785917	0.858292	1.042625	1.230485	0	0	0	0	0	0
1	1.0	-1.746943	1.288474	2.268100	0.969075	0.389147	0	0	0	0	0	0
2	1.0	0.480590	-0.610075	-0.551517	-1.274194	0.389147	0	0	0	0	0	0
3	1.0	0.140797	0.171681	0.153387	-0.244497	-1.293529	0	0	0	0	0	0
4	1.0	0.140797	0.395040	-0.551517	0.251964	0.389147	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...
14995	1.0	1.075116	0.946705	0.858292	0.766813	1.230485	0	0	0	0	0	0
14996	1.0	-0.501035	-1.347425	-1.256422	-1.366131	-0.452191	0	0	0	0	0	0
14997	1.0	1.144980	1.405340	0.153387	1.189725	1.230485	0	0	0	0	0	0
14998	1.0	-1.691546	0.909076	1.563196	1.649411	0.389147	0	0	0	0	0	0
14999	1.0	-0.203112	-0.788428	-1.256422	0.840363	0.389147	0	0	0	0	0	0

15000 rows × 17 columns

In [190…
```python
model_ridotto = sm.Logit(y_train_res, X_train_res_red).fit(disp=0)
```

In [191…
```python
# Calcolo la statistica del Likelihood Ratio Test

lr_stat = 2 * (result.llf - model_ridotto.llf)
df_diff = len(feature_to_exclude)  # gradi di libertà = numero di variabili escluse
p_value = chi2.sf(lr_stat, df_diff)

print(f"Likelihood Ratio Test Statistic: {lr_stat:.4f}")
print(f"Degrees of Freedom: {df_diff}")
print(f"P-value: {p_value:.4f}")

if p_value < 0.05:
    print("Il modello completo è significativamente migliore del modello ridotto: le variabili escluse sono importanti.")
else:
    print("Non c'è evidenza che il modello completo sia migliore: le variabili escluse potrebbero non essere rilevanti.")
```

```
Likelihood Ratio Test Statistic: 104.6659
Degrees of Freedom: 2
P-value: 0.0000
Il modello completo è significativamente migliore del modello ridotto: le variabili escluse sono importanti.
```

In [ ]: